

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 0606

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning **(BL 5)**

Assessment Tool Weightage: 40%

QUESTIONS: 05

Total Marks:100

Annexure A

Coding Challenge

Code of Conduct:

I Gayathri N P (192421362) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Short Answer Test

1, Write a Python program to implement the K-Nearest Neighbour (KNN) algorithm for a small dataset. Use Euclidean distance to find nearest neighbors and classify a test instance. Allow the user to input the value of K and display predicted class labels with accuracy.

Program :

```
import math

# Dataset: [Height, Weight, Class]
data = [
    [150, 50, "A"],
    [160, 55, "A"],
    [170, 65, "B"],
    [180, 75, "B"],
    [155, 52, "A"],
    [175, 70, "B"]
]

# Calculate Euclidean distance
def distance(p1, p2):
    return math.sqrt((p1[0] - p2[0])**2 +
                     (p1[1] - p2[1])**2)

# User input
x = float(input("Enter height: "))
y = float(input("Enter weight: "))
k = int(input("Enter value of K: "))

test = [x, y]

# Find distances
distances = []

for row in data:
    d = distance(test, row)
    distances.append((d, row[2]))

# Sort according to distance
distances.sort()
```

```

# Select K nearest neighbours
neighbors = distances[:k]

print("\nNearest Neighbors:")
for n in neighbors:
    print(n)

# Count class labels
count_A = sum(1 for n in neighbors if n[1] == "A")
count_B = sum(1 for n in neighbors if n[1] == "B")

if count_A > count_B:
    prediction = "A"
else:
    prediction = "B"

print("\nPredicted Class:", prediction)

# Accuracy on training dataset
correct = 0

for test_row in data:
    temp_dist = []

    for row in data:
        if row != test_row:
            d = distance(test_row, row)
            temp_dist.append((d, row[2]))

    temp_dist.sort()
    nearest = temp_dist[:k]

    a = sum(1 for n in nearest if n[1] == "A")
    b = sum(1 for n in nearest if n[1] == "B")

    predicted = "A" if a > b else "B"

    if predicted == test_row[2]:
        correct += 1

accuracy = (correct / len(data)) * 100

print("Accuracy:", accuracy, "%")

output:

```

```

Enter height: 165
Enter weight: 60
Enter value of K: 3

Nearest Neighbors:
(7.0710678118654755, 'A')
(7.0710678118654755, 'B')
(12.806248474865697, 'A')

Predicted Class: A
Accuracy: 100.0 %

```

2. Develop a Python program to implement Locally Weighted Regression. Given a dataset, compute weights based on distance and predict output for a query point. Use a simple kernel function and compare predictions with standard linear regression for better understanding.

Program :

```

import numpy as np

# Dataset
X = np.array([1, 2, 3, 4, 5], dtype=float)
Y = np.array([2, 4, 6, 8, 10], dtype=float)

# Kernel function
def kernel(x, xi, tau):
    return np.exp(-((x - xi) ** 2) / (2 * tau ** 2))

# Locally Weighted Regression
def lwlr(x, tau):
    weights = np.array([kernel(x, xi, tau) for xi in X])

    X_matrix = np.vstack([np.ones(len(X)), X]).T

    W = np.diag(weights)

    theta = np.linalg.pinv(
        X_matrix.T @ W @ X_matrix
    ) @ (X_matrix.T @ W @ Y)

    return theta[0] + theta[1] * x

# Standard Linear Regression
coefficients = np.polyfit(X, Y, 1)

# User input
query = float(input("Enter query point: "))

```

```

tau = float(input("Enter bandwidth (tau): "))

lw_prediction = lwlr(query, tau)

linear_prediction = (
    coefficients[0] * query + coefficients[1]
)

print("\nLocally Weighted Regression Prediction:",
      lw_prediction)

print("Standard Linear Regression Prediction:",
      linear_prediction)

print("\nComparison:")
print("LWR considers nearby points more strongly.")
print("Linear Regression considers all points equally.")

```

output:

```

Enter query point: 3.5
Enter bandwidth (tau): 5

Locally Weighted Regression Prediction: 6.999999999999997
Standard Linear Regression Prediction: 6.999999999999997

Comparison:
LWR considers nearby points more strongly.
Linear Regression considers all points equally.

```

3. Write a Python program to implement a simple Radial Basis Function (RBF) network for regression or classification. Use Gaussian functions as activation units and train the model on a small dataset. Display predicted outputs and analyze how centers and widths affect performance.

Program :

```

import numpy as np

# Training dataset
X = np.array([1, 2, 3, 4, 5], dtype=float)
Y = np.array([2, 4, 6, 8, 10], dtype=float)

# User input
query = float(input("Enter test value: "))
width = float(input("Enter width: "))

# Select centers
centers = np.array([1, 3, 5], dtype=float)

```

```

# Gaussian RBF function
def gaussian(x, center, width):
    return np.exp(-((x - center) ** 2) /
                  (2 * width ** 2))

# Create RBF matrix
R = []

for x in X:
    row = []

    for c in centers:
        row.append(gaussian(x, c, width))

    R.append(row)

R = np.array(R)

# Train output weights
weights = np.linalg.pinv(R) @ Y

# Predict test value
test_rbf = np.array([
    gaussian(query, c, width)
    for c in centers
])

prediction = np.dot(test_rbf, weights)

print("\nCenters:", centers)
print("Weights:", weights)

print("\nPredicted Output:", prediction)

print("\nAnalysis:")
print("Smaller width -> narrow Gaussian functions.")
print("Larger width -> wider Gaussian functions.")
print("Good centers and suitable width improve performance.")

output :

```

```

Enter test value: 3.5
Enter width: 1

Centers: [1. 3. 5.]
Weights: [1.53534931 4.49871527 9.20160174]

Predicted Output: 7.024883540177799

Analysis:
Smaller width -> narrow Gaussian functions.
Larger width -> wider Gaussian functions.
Good centers and suitable width improve performance.

```

4. Create a Python program that implements a basic Case-Based Learning system. Store past cases in a dataset and retrieve the most similar case for a new query using distance measures. Output the solution based on the closest match and explain similarity calculation.

Program

```

import math

# Past cases
cases = [
    ([20, 30], "Low Risk"),
    ([40, 50], "Medium Risk"),
    ([60, 70], "High Risk"),
    ([80, 90], "Very High Risk")
]

# Euclidean distance
def distance(case1, case2):
    return math.sqrt(
        (case1[0] - case2[0]) ** 2 +
        (case1[1] - case2[1]) ** 2
    )

# User input
x = float(input("Enter feature 1: "))
y = float(input("Enter feature 2: "))

query = [x, y]

# Calculate similarity/distance
results = []

for features, solution in cases:
    d = distance(query, features)
    results.append((d, features, solution))

```

```

# Find closest case
results.sort()

closest = results[0]

print("\nSimilar Cases:")

for r in results:
    print("Case:", r[1],
          "Distance:", round(r[0], 2),
          "Solution:", r[2])

print("\nMost Similar Case:", closest[1])
print("Distance:", round(closest[0], 2))
print("Predicted Solution:", closest[2])

print("\nSimilarity Calculation:")
print("Smaller Euclidean distance means higher similarity.")

```

output:

```

Enter feature 1: 45
Enter feature 2: 55

Similar Cases:
Case: [40, 50] Distance: 7.07 Solution: Medium Risk
Case: [60, 70] Distance: 21.21 Solution: High Risk
Case: [20, 30] Distance: 35.36 Solution: Low Risk
Case: [80, 90] Distance: 49.5 Solution: Very High Risk

Most Similar Case: [40, 50]
Distance: 7.07
Predicted Solution: Medium Risk

Similarity Calculation:
Smaller Euclidean distance means higher similarity.

```

5. Create a Python program to implement the K-Means clustering algorithm. Initialize cluster centers, assign points to clusters, and update centroids iteratively. Display final clusters and visualize them using plots. Analyze how the number of clusters and initialization methods affect clustering results.

Program :

```

import numpy as np
import matplotlib.pyplot as plt

# Dataset
points = np.array([
    [1, 1],

```



```

[1, 2],
[2, 1],
[8, 8],
[9, 8],
[8, 9]
], dtype=float)

# User input
k = int(input("Enter number of clusters: "))

# Initial centroids
centroids = points[:k].copy()

for iteration in range(10):

    # Assign points to nearest centroid
    clusters = []

    for point in points:
        distances = [
            np.linalg.norm(point - centroid)
            for centroid in centroids
        ]

        cluster = np.argmin(distances)
        clusters.append(cluster)

    clusters = np.array(clusters)

    # Update centroids
    new_centroids = []

    for i in range(k):

        cluster_points = points[clusters == i]

        if len(cluster_points) > 0:
            new_centroid = cluster_points.mean(axis=0)
        else:
            new_centroid = centroids[i]

        new_centroids.append(new_centroid)

    new_centroids = np.array(new_centroids)

    # Stop if centroids do not change

```

```

if np.allclose(centroids, new_centroids):
    break

centroids = new_centroids

# Display clusters
print("\nFinal Centroids:")
print(centroids)

for i in range(k):
    print("\nCluster", i + 1)
    print(points[clusters == i])

# Visualization
for i in range(k):
    cluster_points = points[clusters == i]

    plt.scatter(
        cluster_points[:, 0],
        cluster_points[:, 1],
        label="Cluster " + str(i + 1)
    )

plt.scatter(
    centroids[:, 0],
    centroids[:, 1],
    marker="X",
    s=200,
    label="Centroids"
)

plt.xlabel("X")
plt.ylabel("Y")
plt.title("K-Means Clustering")
plt.legend()
plt.show()

print("\nAnalysis:")
print("Different K values produce different clusters.")
print("Different initial centroids can produce different results.")
print("Good initialization usually gives better clustering.")

```

output:

Enter number of clusters: 2

Final Centroids:

```
[[1.33333333 1.33333333]
 [8.33333333 8.33333333]]
```

Cluster 1

```
[[1. 1.]
 [1. 2.]
 [2. 1.]]
```

Cluster 2

```
[[8. 8.]
 [9. 8.]
 [8. 9.]]
```

Analysis:

Different K values produce different clusters.

Different initial centroids can produce different results.

Good initialization usually gives better clustering.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	15	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	20	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand